



Autonomous navigation of UAV in complex environment : a deep reinforcement learning method based on temporal attention

Shuyuan Liu^{1,2} · Shufan Zou³ · Xinghua Chang^{1,2} · Huayong Liu^{1,2} · Laiping Zhang^{1,2} · Xiaogang Deng^{1,2,4}

Accepted: 11 September 2024 / Published online: 14 January 2025

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2025

Abstract

With the increasing demand for Unmanned Aerial Vehicles (UAVs) in both military and civil applications, the ability for UAVs to automatically avoid obstacles and navigate to specific destinations has been receiving growing attention. However, most current methods focus on environments where global information is available or both destination and obstacles are static, which are not suitable for dense, dynamic, complex real-time tasks. Therefore, we propose a novel autonomous navigation method based on Deep Reinforcement Learning (DRL), which is suitable for more complex environments. Based on the Soft Actor-Critic (SAC) algorithm, this method incorporates changes of the state space into network input with a temporal attention mechanism, which allows UAVs to adaptively extract key information from historical environments while maintaining sensitivity to the current environment. We establish a visualized two-dimensional navigation task environment and design different simulation tests to evaluate its the performance and generalization. Results show that compared to baselines, our algorithm can achieve higher average rewards and more stable convergence after training in a static multi-obstacle environment, and can demonstrate better performance in environments featuring multiple obstacles of varying numbers, sizes, and speeds, thereby achieving a balance between task completion efficiency and security.

Keywords Deep reinforcement learning · Unmanned aerial vehicle · Autonomous navigation · Temporal attention · Obstacle avoidance

1 Introduction

In recent years, the Unmanned Aerial Vehicle (UAV) has shown good application prospects in military and civil fields due to its high flexibility, strong maneuverability, low risk cost and low labor cost [1–7]. Therefore, how to ensure that the UAV can reach the destination safely and efficiently under the premise of avoiding various obstacles has received more and more attention [8].

Currently, UAVs typically utilize on-board cameras, laser rangefinders, or other sensors to obtain information to execute the navigation task [9]. According to the amount of information obtained by the UAV, the traditional path planning can be divided into global path planning and local path planning [10]. Global path planning is usually to find an optimal or suboptimal path from the starting point to the destination under the condition that the global information is known and the task environment is static. The commonly used global path planning methods are A* [11], RRT [12], genetic algorithm [13], RRT* [14], particle swarm optimization [15, 16] and so on. The common point of these methods is to have the complete information of the task in advance and establish an abstract global environment map model, so as to obtain the path of the UAV by means of search or sampling. Local path planning includes more real-time and dynamic planning method, such as dynamic window method [17], time elastic band [18], artificial potential field method [19] and so on. Although traditional path planning algorithms can solve some simple navigation tasks, they still face many challenges, such as difficulty in obtaining global environment

✉ Xinghua Chang
cxh_cardc@126.com

- ¹ National Key Laboratory of Fundamental Algorithms and Models for Engineering Simulation, Sichuan University, Chengdu 610277, China
- ² College of Computer Science, Sichuan University, Chengdu 610065, China
- ³ College of Aerospace Science and Engineering, National University of Defense Technology, Changsha 410000, China
- ⁴ Academy of Military Sciences, Beijing 10019, China

information, sensor accuracy error, environment noise disturbance, and the contradiction between the accuracy of map discretization and memory [10]. Furthermore, the increasing complexity of the task environment also makes the computational cost of the algorithm greatly grow [8].

In recent years, the rapid development of artificial intelligence has made it possible to solve the problem of autonomous navigation of UAVs by using deep learning, a technology with strong representation ability and non-linear fitting ability. Within it, deep reinforcement learning (DRL), an artificial intelligence paradigm, gains more attention due to its model-free, end-to-end learning capability and low requirement for sensor accuracy [10]. As one of the paradigms that are expected to realize artificial general intelligence (AGI) [20], DRL has many advantages over traditional algorithms in path planning or autonomous navigation problems. It can not only adaptively optimize strategies by trial and error in complex environments, but also conduct end-to-end learning directly from raw sensor data, thereby enhancing the robustness of the control strategies. Furthermore, the non-linear fitting capability of DRL enables a continuous mapping from the environment to action execution, which allows it to demonstrate notable generalization capabilities across different environments and resolve some complex problems that traditional algorithms find challenging to handle.

This paper aims to solve the navigation problem of UAVs in complex and dynamic environments by using DRL. The main contributions of this paper are as follows:

(1) We propose a novel SAC algorithm framework based on temporal attention. This framework can extract the key historical information in the environment and fuse it with the current state, so that the UAV can make more appropriate decisions to complete the navigation task safely and efficiently.

(2) By modeling the autonomous navigation task, we design a composite reward function with multiple sub-goals. This enables the UAV to receive appropriate reward feedback after executing each action, effectively addressing the sparse reward problem in DRL [21].

(3) We construct a visualized two-dimensional continuous simulation platform to minimize the disparity between simulation and reality. We conduct comparative experiments in this simulation environment and select original SAC as the basic algorithm. Then, we verify the effectiveness and generalization of the proposed algorithm by comparing with the baseline and conducting ablation experiments.

The other contents of this paper are as follows: In Section 2, we summarize the related work of applying DRL to UAV navigation tasks. Section 3 gives a brief introduction to the preliminaries, including the background knowledge of MDP and DRL, navigation task modeling, as well as the state space and action space of the UAV. In Section 4, we

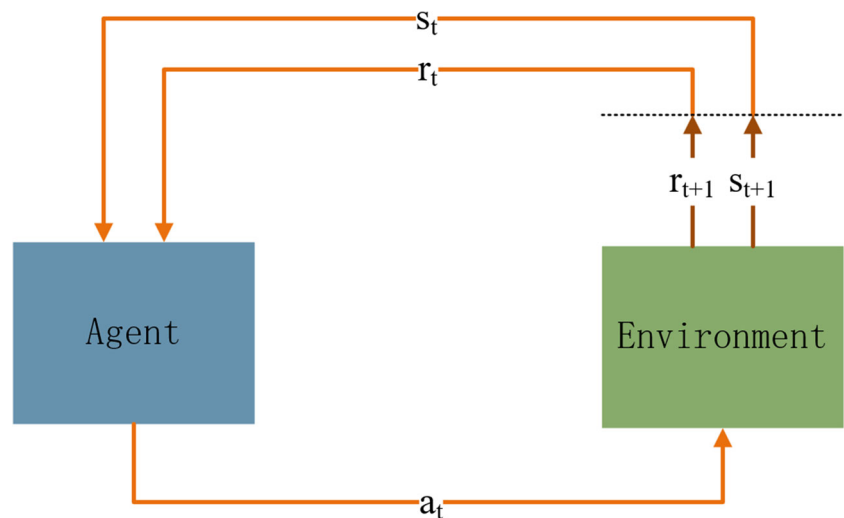
deliver a detailed exposition of the proposed method from the aspects of normalization, neural network architecture and reward function design. Section 5 describes the specific environment construction and training settings, and discusses the results of the training and simulation tests. Finally, Section 6 summarizes the contributions of this paper and looks forward to some future work.

2 Related work

Ever since Mnih et al. [22] proposed the concept of DRL and the Deep Q-Network (DQN), there has been an increasing trend to apply DRL to UAV autonomous navigation, which leads to a degree of progress [23]. Huang et al. [24] proposed a novel approach that integrates DQN with multiple-input multiple-output (MIMO) technology, enabling UAV to make decisions based on received signal strength. Moon et al. [25] applied DQN to path planning, utilizing received signal strength as a reward function to guide UAV flight. However, as the task environments become increasingly complex, DQN-based methods characterized by discrete action space exhibit limitations because they may lead to a curse of dimensionality when facing continuous action space. With the introduction of the Actor-Critic (AC) architecture, various policy-based DRL algorithms have been employed to address the challenges of continuous action spaces in navigation tasks. Wang et al. [26] proposed a concept called nonexpert helpers (LwH) to address the challenges posed by sparse rewards, and they validated the effectiveness of this component using the Asynchronous Advantage Actor-Critic (A3C) algorithm. Zhang et al. [27] established an actor network that is more suitable for task requirements by using two sub-networks that focus on the perception-related state and the target-related state, respectively. They applied the method to Deep Deterministic Policy Gradient (DDPG) and demonstrated it can outperform baselines in terms of convergence rate, navigation performance, and generalization ability within the constructed task environment.

Despite these developments, most methods based on DRL, rooted in the Markov Decision Process (MDP), often fail to consider historical information, which is crucial in highly complex environments. This oversight can lead to decision-making errors and the freezing robot problem, where the UAVs struggle to determine optimal actions. To solve this problem, recent studies have attempted to incorporate well-designed or temporal features into the state space. Li et al. [28] explored the complex fusion of state and action and proposed a three-channel fusion mechanism based on DDPG to aid autonomous navigation for UAVs. Zhang et al. [29] proposed a two-stream network structure based on Twin Delayed Deep Deterministic Policy Gradient (TD3) to extract envi-

Fig. 1 The diagram of Deep Reinforcement Learning



ronment observation features, so as to obtain the approaching or receding state of obstacles within the sensor range. Xue et al. [30] proposed Fast Recurrent SVG(0), which utilizes Long Short Term Memory networks(LSTM) for temporal feature extraction in the state space, to improve sampling efficiency for UAV continuous navigation in partially observable environments. In subsequent work [31], they attempted to integrate classical motion planning algorithms with DQN. After extracting temporal features of the state space using LSTM, they generated local goals for the lower-level classical planner to produce smooth and safe trajectories to the target.

It is worth noting that although these methods enable UAVs to obtain more environment-related information through manual feature design and temporal feature processing, there are still some limitations. Manual feature design may fail to capture the full complexity of complex environments, leading to suboptimal representations. Similarly, simple RNN-based methods, though effective in handling sequential data, often struggle with long-term dependencies and the prioritization of relevant historical information. These limitations can result in UAVs making poor decisions, where they are unable to navigate effectively due to an overload of irrelevant or redundant information. Additionally, the simulation environments involved in most of the above studies are relatively simple, that is, the obstacles are limited to sparse, static or low-speed settings, and do not consider more dense, high-speed, highly random obstacles that may collide with each other.

Based on the analysis of the aforementioned problems, we propose a novel autonomous navigation method based on the Soft Actor-Critic (SAC) algorithm, which aims to augment the performance of UAVs in executing autonomous navigation tasks within complex environments.

3 Preliminaries

3.1 DRL and MDP

Deep reinforcement learning (DRL) is based on Markov decision process (MDP) to describe the closed-loop interaction process between the agent and the environment [32]. As shown in Fig. 1, agents are entities that are needed to complete a specific task. They only need to consider the current state and select actions in the environment, which will affect the evolution of the environment situation and the generation of environment rewards.

In this process, the interaction between the agent and the environment is usually defined by a quadruple (S, A, P, R) . Within it, S is the state space, including all possible environment states $s \in S$ that can be observed by the agent. A is the action space, where each action $a \in A$ represents the behavior that the agent can take in the environment. P is the state transition probability function $P(s_{t+1} | s_t, a_t)$, which defines how much probability the system will reach the new state s_t by selecting action a_t under state s_t . R is a reward function $R(s_t, a_t, s_{t+1})$, which is responsible for giving feedback to the agent on the process of taking action a_t from state s_t and reaching a new state s_{t+1} . And the agent can use the feedback to optimize the behavior strategy. The optimization process of this strategy is generally completed by the iteration of the Bellman Equation. The usual form of the Bellman equation of the action-value function is:

$$Q(s_t, a_t) = r(s_t, a_t) + \gamma \cdot E_{s_{t+1}, a_{t+1}} [Q(s_{t+1}, a_{t+1})], \quad (1)$$

where γ is the discount factor, which is used to adjust the current value of the future reward, and Q is the action-value function, which is used to describe the expected cumulative

reward after the action a is executed in the state s under the strategy π . After describing the UAV navigation problem as MDP, how to make the UAV safely and efficiently avoid obstacles and reach the destination is transformed into how to select the action that maximizes the Q -value in each state, which is also the problem that DRL tries to solve.

3.2 Problem formulation

3.2.1 Task modeling

As shown in Fig. 2, we design a two-dimensional environment of $WIDTH \times HEIGHT$, with gravity of G and vertical downward.

Within it, *FIELD1* and *FIELD3* are the target and starting point generation areas, that is, the target and starting point are randomly generated in the blue or green vertical area at both ends. *FIELD2* is the obstacle generation area, that is, obstacles are randomly generated in the middle orange vertical area. The UAV is regarded as a particle, and due to some limitations of sensors like visual cameras and radars, such as sensitivity to changes in ambient light and the high cost of deployment and maintenance, we choose a circular infrared rangefinder with a radius of INF_R to measure the distance between the UAV and obstacles. The obstacle is randomly set to a square with side length OBS_L or a circle with radius OBS_R . The target is set to a circle with a radius of TAR_R , representing the destination. Furthermore, the maximum speed of the UAV is set to MAX_V , the obstacle initial speed is OBS_V , the time step is $TIME_STEP$, the maximum number of time steps for a single task episode is MAX_STEP .

In this environment, the goal of UAV autonomous navigation tasks can be modeled as:

$$\begin{cases} \min \sum_{i=0}^T \|p_{i+1} - p_i\|_2 \\ s.t. \ d_{min} > 0 \\ p_0 = [x_{origin}, y_{origin}], p_T = [x_{target}, y_{target}] \end{cases}, \quad (2)$$

where T represents the last time step of a round of the task, x_{origin} and x_{target} represent the horizontal coordinates of the origin and target respectively, y_{origin} and y_{target} represent the vertical ordinates of the origin and target respectively, p_0 and p_T represent the UAV's position of the origin and target, and d_{min} represents the minimum distance between the UAV and the obstacle in the whole navigation task. In general, the goal of the autonomous navigation can be expressed as: reaching the target as quickly as possible from a starting point, and not colliding with obstacles in the whole process.

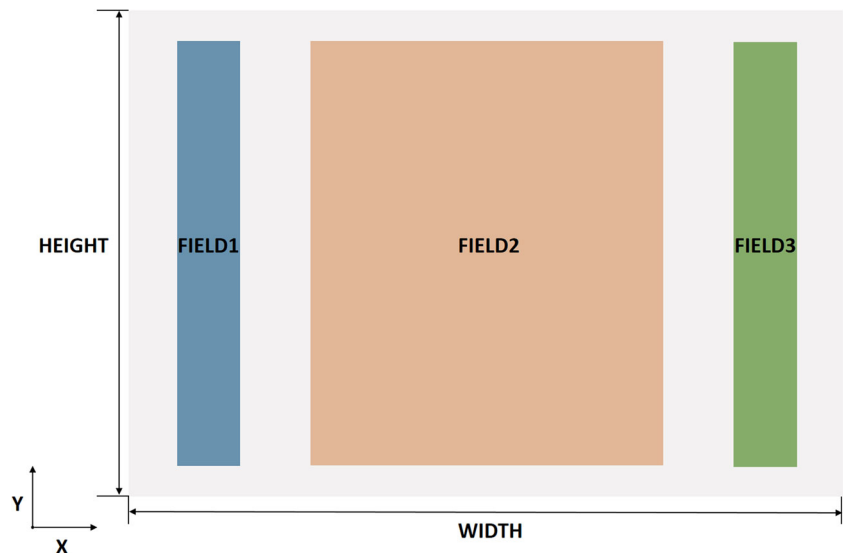
3.2.2 State space

In the training process of the DRL algorithm for UAV autonomous navigation algorithm, the setting of the state space plays a vital role which is a multi-dimensional abstract space, and each dimension provides important information about the current task environment of the UAV. The reasonable definition of the state space can make the algorithm model more accurately grasp the complexity of the navigation task and help the UAV to select the best action according to the current state.

The state space of the UAV navigation task designed by us is composed of *selfInfo* and *obsInfo*, that is:

$$S = [selfInfo, obsInfo], \quad (3)$$

Fig. 2 The diagram of navigation environment



where *selfInfo* represents the self-information measured by the global positioning system (GPS) and gyroscope in real time, that is:

$$\text{selfInfo} = [\text{abs_x}, \text{abs_y}, \text{rel_x}, \text{rel_y}, \text{vel_x}, \text{vel_y}], \quad (4)$$

where (*abs_x*, *abs_y*) is the absolute position of the UAV with the lower left corner of the simulation environment as the origin, so that it can know its own coordinates in the environment and avoid going out of the boundary. At the same time, in order to allow the UAV know the location of the target and accelerate the convergence of the algorithm, we design the relative position (*rel_x*, *rel_y*) with the target as the origin. Then, (*vel_x*, *vel_y*) is set to let the UAV know its own motion information along the X-axis and Y-axis respectively.

Next, *obsInfo* represents the information about obstacles, which is measured in real time by the infrared rangefinder carried by the UAV. The method of measurement is shown in Fig. 3. The infrared rangefinder, with the UAV as its origin, emits infrared rays to a maximum length of *INF_R* at every $\pi/6$ rad interval, aiming to detect whether there are obstacles in its path. To sum up, *obsInfo* is defined as:

$$\text{obsInfo} = [o_1, o_2, \dots, o_{12}],$$

$$\text{where } o_j = \begin{cases} \text{INF_R} & \text{if no obstacles detected} \\ \text{detected_distance} & \text{otherwise} \end{cases}, \quad j \in [1, 12]. \quad (5)$$

In other words, if the detected distance of a ray equals *INF_R*, it indicates that no obstacle has been detected by the ray. However, if the detected distance is less than *INF_R*, it

means that an obstacle has been identified by the ray. Furthermore, the closer this value is to zero, the closer the detected obstacle is to the UAV.

3.2.3 Action space

In order to reduce the errors introduced by the coupling of low-level controllers during the actual deployment of the algorithm, and to further demonstrate the robustness of the algorithm we propose, we express the UAV's action space directly as two main parameters: the magnitude and direction of the thrust. Therefore, the action space of the UAV is:

$$A = [F, \theta], \quad F \in [0, \text{MAX_F}], \theta \in [0, 2\pi], \quad (6)$$

where *F* represents the magnitude of the thrust applied to the UAV, within the range of $[0, \text{MAX_F}]$, and θ represents the direction of the thrust, within the range of $[0, 2\pi]$. Together, these two parameters determine the motion of the UAV.

obstacle

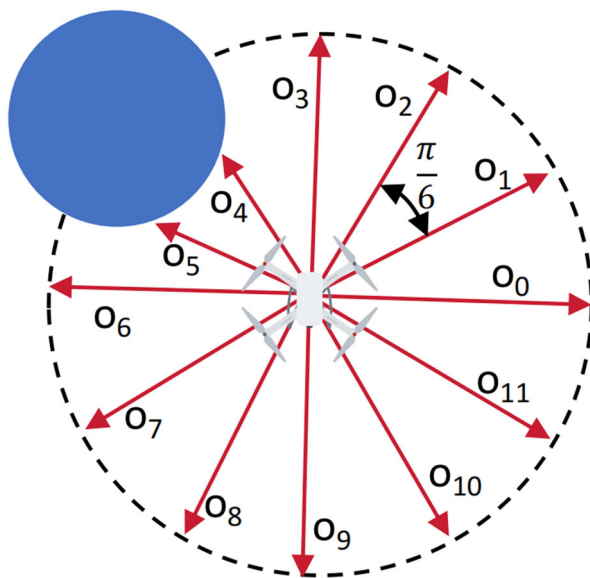


Fig. 3 The diagram of the obstacle measurement

4 Proposed method

4.1 Normalization

In the training process of DRL, the state space and action space often involve various data types, which may vary in scale and range. Such characteristics can lead to an unstable training process and slow convergence speed, even potentially hindering the convergence of the algorithm. Therefore, normalizing these data can enhance the speed and the stability of convergence, thereby improving the performance of the DRL algorithm. Given these considerations, we normalize the state space and action space as follows:

$$\begin{aligned} x &= \frac{x}{\text{WIDTH}}, \quad x = \text{abs_x}, \text{rel_x}, \\ y &= \frac{y}{\text{HEIGHT}}, \quad y = \text{abs_y}, \text{rel_y}, \\ v &= \frac{v}{\text{MAX_V}}, \quad v = \text{vel_x}, \text{vel_y}, \\ o_j &= \frac{o_j}{\text{INF_R}}, \quad j \in [1, 12], \\ F &= \frac{F}{\text{MAX_F}}, \\ \theta &= \frac{\theta}{2\pi}. \end{aligned} \quad (7)$$

4.2 Soft Actor-Critic

In Section 2, we model the UAV navigation task in a continuous state space and action space in order to make the trained algorithm more suitable for deployment in the actual environment. In this paper, we use SAC [33], noted for its stronger exploration capabilities in complex environments compared to other DRL algorithms that handle continuous action spaces, to handle such a task.

SAC belongs to the AC architecture in DRL [34], which is an important paradigm in DRL. It effectively combines the advantages of value-based methods and policy-based methods. In the AC architecture, the neural network is divided into two parts, one is the Actor and the other is the Critic. The Actor represents the strategy π , which will output an action at according to the current state s_t , that is $a_t = \pi(s_t)$. On the other hand, the Critic represents the action-value function, which will evaluate the value of the action at selected by the Actor, that is $Q(s_t, a_t)$. Then, the Actor will optimize the strategy according to the evaluation to output a better action in the subsequent round. Specifically, the time difference (TD) method is usually used to update the network parameters of the Critic. The idea is to use the difference between the sum of the Q -value of the next state s_{t+1} and the reward r_t and the Q -value of the current state s_t to update the network parameters. The objective function of the Critic is:

$$L = E[(Q(s_t, a_t) - (r_t + \gamma \cdot Q'(s_{t+1}, a_{t+1})))^2], \quad (8)$$

where γ is the discount factor, which is used to adjust the current value of future rewards. The purpose of the Actor is to obtain the maximum expected total reward, it usually uses the policy gradient method to update the network parameters. The objective function of the Actor is:

$$J = E_{s \sim p_\pi, a \sim \pi}[Q(s, a)], \quad (9)$$

where p_π is the state distribution decided by strategy π .

Compared with other algorithms (PPO, DDPG, TD3, etc.) that also belong to the AC architecture, the most prominent feature of SAC is to integrate the maximum entropy principle into the algorithm design, which enhances the agent's ability to explore the environment. The Q -value of SAC is:

$$Q = r(s_t, a_t) + \gamma \cdot E_{s_{t+1}, a_{t+1}}[Q(s_{t+1}, a_{t+1}) - \alpha \cdot \log(\pi(a_{t+1}|s_{t+1}))], \quad (10)$$

where $-\log(\pi(a_{t+1}|s_{t+1}))$ represents the negative log-probability of action a_{t+1} under the strategy π in state s_{t+1} , reflecting the uncertainty and exploration ability of the current strategy. The entropy of the strategy is typically calculated as the expectation of these terms over all possible actions. And α is the temperature coefficient that adjusts the

importance of strategy entropy. Compared to the Q -value function of the original AC architecture as outlined in (1), SAC's Q -value function makes the policy learning objective under SAC accomplish more than simply maximizing the expected cumulative rewards. It also aims to reinforce the policy's exploratory capability, thus directing it to focus on a wider range of potentially beneficial actions as opposed to a single optimal action during the learning process. This approach enhances the agent's accuracy in predicting future rewards, thereby boosting the algorithm's capacity for generalization and robustness. Simultaneously, in a continuous action space, SAC employs a stochastic policy as opposed to the deterministic policy which invariably selects the same action given a specific state. The stochastic policy produces the action's probability distribution from which an action is sampled to execute. This approach offers the agent a broader exploration space, thereby elevating the possibility of locating a globally optimal solution in a complex environment. Specifically, SAC utilizes a reparameterization technique to acquire the action's probability distribution. The Actor network concludes with two separate linear layers. One outputs the mean (μ) of the Gaussian distribution of the action, while the other outputs $\log_std$, the action's logarithm of the standard deviation of the Gaussian distribution. Consequently, for each state, these two parameters can be used to sample an action from the Gaussian distribution. The application of the reparameterization technique enables the neural network's learning process to remain gradient-descent tractable (i.e., differentiable). Lastly, to alleviate the issue of Q -value overestimation in DRL, SAC typically employs a double Q -network [35] with target network [36], facilitating more rapid and stable training progress.

In the initial stage of training, the agent explores the environment by randomly sampling from the action space, and the experiences (s_t, a_t, r_t, s_{t+1}) are stored in an experience replay buffer. Once sufficient experiences have been accumulated, SAC is employed to update the parameters of the neural network. The workflow of SAC is shown in Fig. 4. Specifically, when a tuple of experience (s_t, a_t, r_t, s_{t+1}) is extracted for updating, s_{t+1} is input into the Actor network to get the subsequent action a_{t+1} , yielding the pair (s_{t+1}, a_{t+1}) , and then each Critic_Target network generates a Q -value, representing the expected return of taking action a_{t+1} in state s_{t+1} . The lower of these two Q -values is utilized to compute the TD target. The TD target is computed as:

$$y(r_t, s_{t+1}) = r_t + \gamma \cdot \min_{i=1,2} (Q_{t-i}(s_{t+1}, a_{t+1})) - \alpha \cdot \log(\pi(a_{t+1}|s_{t+1})), \quad (11)$$

where α is the temperature coefficient and γ is the discount factor. Following the computation of the TD target, the Q -value errors for each of the Critic networks are calculated.

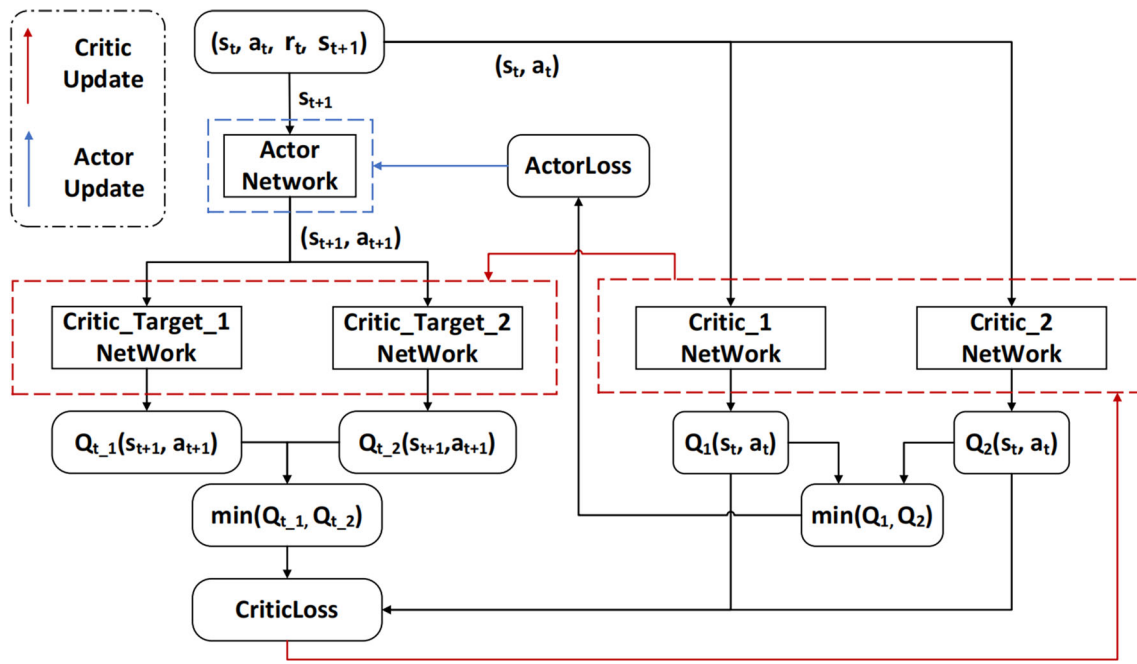


Fig. 4 The workflow of SAC

These errors are then utilized to update the Critic networks. The loss function is defined as:

$$\text{CriticLoss} = (Q_i(s_t, a_t) - y(r_t, s_{t+1}))^2, i = 1, 2. \quad (12)$$

Then, the Q -value of the Critic network is used to update the Actor network, and the loss function is:

$$\text{ActorLoss} = \alpha \cdot \log(\pi(a_t | s_t)) - \min_{i=1,2} (Q_i(s_t, a_t)). \quad (13)$$

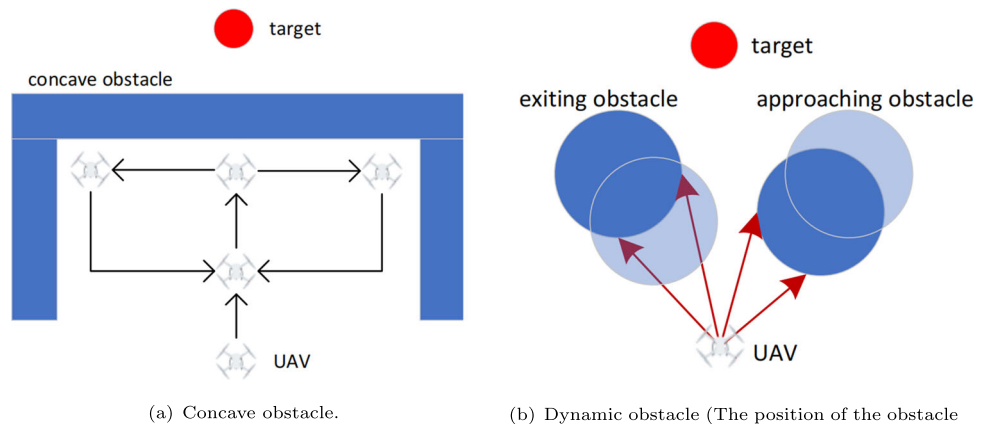
After the updates to the Critic and Actor networks, a soft update method is applied to the Target_Critic networks.

4.3 Temporal attention network

Owing to its ability to explore the environment, SAC is apt for managing UAV navigation tasks in complex environments that have continuous state space and action space. However, as it selects and executes actions based solely on the environment information perceived at the current moment, it is unable to access historical environment information. This limits its ability to observe complex, dense, and dynamic environments, resulting in deviations in agent's decision-making.

As Fig. 5(a) illustrates, when the UAV encounters a large concave or concave-like obstacle, it tends to wander among obstacles, leading to a mission timeout. This issue, which

Fig. 5 The avoidance dilemma in complex environments



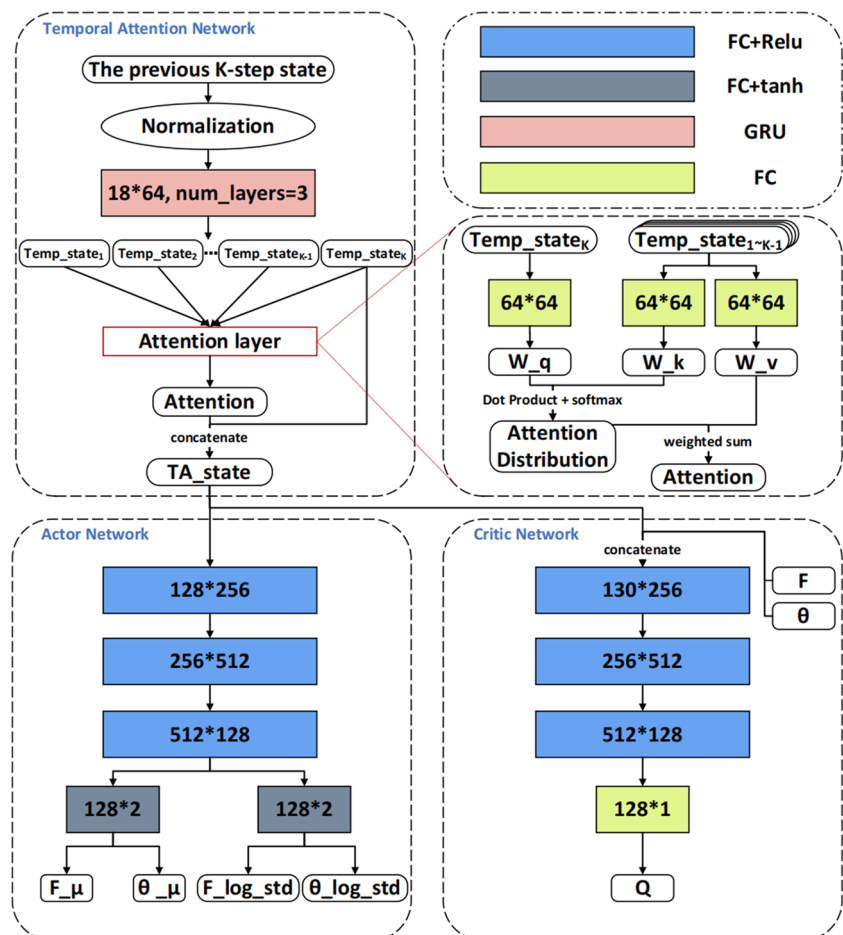
is called the target obstacle dilemma [37] or freezing robot problem [38], stems from the fact that the UAV can only observe the current environment and struggles to effectively gather pertinent information from historical observations. Consequently, it fails to recall that it has previously lingered in the same location. Furthermore, as shown in Fig. 5(b), when faced with dense and dynamic obstacles, it is crucial for the UAV to extract valuable movement information from these obstacles. This would facilitate more informed decision-making, such as whether to appropriately approach the obstacle that is moving away to reach the target more quickly, or to evade the approaching obstacle promptly. However, original SAC can't extract such temporal information relying solely on the obstacle information provided by the infrared rangefinder at the instantaneous moment. This limitation can lead to unsuitable action outputs from SAC, culminating in the deviation of the UAV's operational mode to accomplish the autonomous navigation task.

In order to overcome this challenge, the introduction of recurrent neural network (RNN) has marked improvements [30]. However, RNN suffers from an inherent forgetting issue, where it struggles to retain older information while processing new inputs. Another idea is to obtain the spatial

transformation information of obstacles by manually designing the features of the dynamic information of obstacles [29], but it does not consider the dynamic information of the UAV itself and lacks longer time reasoning ability. In view of this, we propose a temporal attention network designed to extract temporal features from the environment during UAV navigation tasks, which allows the UAV to focus on key details within its historical state and make more accurate decisions based on the current time step. The network structure is illustrated in Fig. 6.

In our designed network structure, the state input comprises a time series. This series is composed of environment information from the current time step paired with that from the preceding K time steps, where K is a hyperparameter that determines how far back the state information is considered. After normalizing this information from the K previous time steps, it is input into a gated recurrent unit network (GRU) to extract temporal features at each step. These extracted temporal features are subsequently passed through an attention layer. Within the attention layer, the temporal feature vector from the current step is retrieved. A dot-product attention operation then takes place with the feature vectors from each preceding time step and the feature vector from the current

Fig. 6 The network structure of SAC with temporal attention



step. Next, these vectors are passed through a softmax layer to obtain the attention distribution, which is then used along with the current time step feature vector to compute the attention. In order for the agent to focus on both the current and historical states concurrently, the calculated attention is concatenated with the feature vector from the current time step, yielding TA_state .

As the input of the Actor network, the TA_state is processed through three successive fully connected (FC) neural network layers, each followed by a rectified linear unit (ReLU) activation function. These layers have node counts of 256, 512, and 128, respectively. Then, the vectors resulting from these three layers are input into two separate FC layers, each comprised of two nodes. Both of these layers are then connected to a hyperbolic tangent (tanh) activation function, ensuring that the respective mean value, μ , and logarithmic standard deviation, log_std , of the output actions remain within the range $[-1, 1]$. For the Critic network, we employ μ and log_std to sample the action space, thereby obtaining actions F and θ . These actions are concatenated with the TA_state and are processed through three FC+ReLU network layers with 256, 512 and 128 nodes respectively. After this, the vector is mapped to the Q -value through an additional FC layer.

The algorithm of the method we propose is shown in Algorithm 1.

4.4 Reward design

In the training process of DRL algorithms, the reward function plays a pivotal role. It supplies real-time feedback on the agent's selected actions during the learning phase, thereby guiding the agent's behavior. As a quantitative measure of the agent's actions, the output of the reward function sets the course for the optimization of its behavior strategy. During the learning phase, an ideal reward function should not only enable the agent to identify the optimal strategy promptly and efficaciously, but also encourage it to explore unknown environments to a certain degree. For this study, the reward function's purpose is to guide the UAV to efficiently avoid obstacles while promptly reaching the target. Accordingly, we decompose the UAV autonomous navigation task into

Algorithm 1 SAC based on temporal attention.

```

1: Initialize Critic_1 and Critic_2 with parameters  $\omega_1$  and  $\omega_2$ ,
   Critic_Target_1 and Critic_Target_2 with parameters  $\omega_1^- = \omega_1$  and
    $\omega_2^- = \omega_2$ , and Actor with parameter  $\theta$ .
2: Initialize experience replay buffer  $D$  with size  $N$ , temperature coefficient
    $\alpha$ , and soft update coefficient  $\tau$ .
3: for  $episode = 1$  to  $E$  do
4:   Obtain the initial state of the environment.
5:   for  $time\_step = 1$  to  $T$  do
6:     Process the previous  $K$ -step state through Temporal Attention
       Network and obtain  $TA\_state_t$ , then select  $a_t = \pi_\theta(TA\_state_t)$ .
7:     Execute  $a_t$ , obtain a new state  $s_{t+1}$  and a reward  $r_t$ .
8:     Store experience  $(\{s_i\}_{i=t+1-K, \dots, t}, a_t, r_t, \{s_i\}_{i=t+2-K, \dots, t+1})$ 
9:     if number of experiences ==  $N$  then
10:      Sample an experience, input  $\{s_i\}_{i=t+2-K, \dots, t+1}$  into Temporal
        Attention Network and obtain  $TA\_state_{t+1}$ , then select  $a_{t+1} = \pi_\theta(TA\_state_{t+1})$ .
11:      Acquire  $Q_{t,j}$  from Critic_Target_1,  $i=1,2$ .
12:      Calculate TD target as (11), use it to update Critic_1 and
        Critic_2 as (12).
13:      Update the Actor as (13).
14:      Update  $\alpha$  with auto mode.
15:      Update Critic_Target_1 and Critic_Target_2 with  $\tau$ .
16:    end if
17:  end for
18: end for

```

four sub-goals: reaching the target, minimizing the time cost, avoiding obstacles, and staying within environment boundaries. Guided by these objectives, we design a compound reward function, which is detailed in Table 1.

The first is the distance reward, designed to penalize the UAV for unnecessary waiting in the environment and incentivize task completion. We use the distance between the UAV and the target as a penalizing factor, where the further the UAV is from the target, the greater the penalty incurred, and η is a negative hyperparameter that controls the severity of the penalty. Meanwhile, a significant reward is given when the UAV reaches the target. Secondly, the shaping reward promotes the UAV to minimize the distance to the target while executing the navigation task. We calculate this reward based on the difference between the UAV's distance to the target at the current and that at the previous time step. The quicker the UAV moves to the target, the larger reward is given, and vice versa. β is a positive hyperparameter that controls the strength of the reward. Next, we design an avoidance penalty

Table 1 Reward function settings

Description	Reward function
Distance reward	$r_{distance} = \begin{cases} \eta \cdot \ln(distance + 1), & \text{if } distance > TAR_R \\ 20, & \text{if } distance == TAR_R \end{cases}$
Shaping reward	$r_{shaping} = \beta \cdot (last_distance - distance)$
Avoidance penalty	$r_{avoidance} = \begin{cases} \psi \cdot e^{\sigma \cdot \min(obsInfo)}, & \min(obsInfo) < \frac{INF_R}{5} \\ -20, & \text{if collides with obstacles} \end{cases}$
Boundary penalty	$r_{boundary} = -20, \text{ if out of boundaries}$

to guide the UAV to avoid obstacles. Specifically, when the minimum value of the infrared rangefinder data (*obsInfo*) is less than one-fifth of the safe distance (*INF_R*), a penalty is imposed—greater proximity to obstacles results in larger penalty values. ψ and σ theta are two negative hyperparameters that control the severity of the penalty. Furthermore, if the UAV collides with an obstacle, the task is judged to fail and a large penalty is given. Finally, when it comes to the boundary penalty, if the UAV flies out of the environment boundary, we regard it as a failure of the task and give it a large penalty.

In conclusion, the above four reward functions based on sub-goals are directly summed to form a compound reward function, which works to guide the autonomous navigation task of the UAV. The reward function that we have designed can be articulated as follows:

$$R = r_{distance} + r_{shaping} + r_{avoidance} + r_{boundary}. \quad (14)$$

5 Experiments and analysis

5.1 Simulation environment and training settings

In order to simulate a navigation task environment replete with complex, dense, and dynamic obstacles, we utilize Gym to fabricate a visualized two-dimensional continuous simulation platform. As a framework for DRL research, Gym not only offers multiple settings for researchers to evaluate and refine their DRL algorithms, but also furnishes a comprehensive environment interface to construct a bespoke environment.

Figure 7 presents a typical instance of the simulation environment that we have created, in which the red solid circle signifies the target, while the red hollow circle and its center represent the boundary of the infrared detector and the UAV respectively. The black solid circle and the black solid square are used to indicate obstacles. Comprehensive specifics

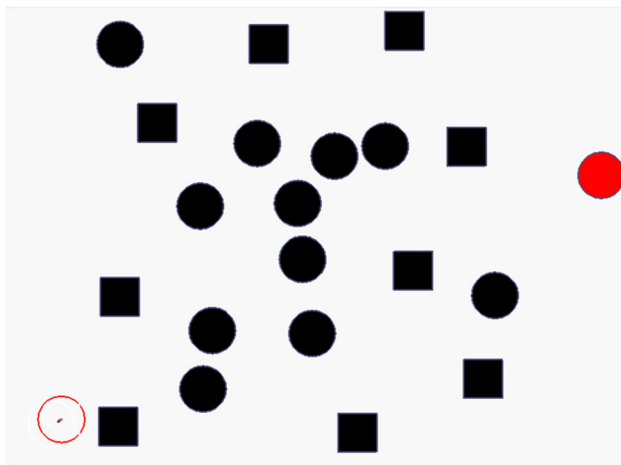


Fig. 7 Typical example of the simulation environment

Table 2 Simulation environment parameter settings

Symbols	Descriptions	Values
WIDTH	Width of the environment	800
HEIGHT	Height of the environment	600
INF_R	Radius of the infrared ranger	30
TAR_R	Radius of the target	30
MAX_V	Maximum speed of the UAV	10
TIME_STEP	Time step of the environment	0.1
MAX_STEP	The maximum step per episode	2000
MAX_F	The maximum force applied to the UAV	40
G	Gravity of the environment	10
OBS_NUM	Number of obstacles	Settable
OBS_V	Magnitude of the initial velocity of obstacles	Settable
OBS_R	Radius of the circular obstacle	Settable
OBS_L	Side length of the square obstacle	Settable

regarding the parameters of the simulation environment are detailed in Table 2. Among them, the environment parameters related to obstacles, namely *OBS_NUM*, *OBS_V*, *OBS_R*, *OBS_L*, are settable, while other parameters remain fixed, which means the number, initial speed, and size of obstacles can be adjusted according to experimental needs.

We implement all algorithms on a workstation using an NVIDIA GeForce RTX 4070 and determine the hyperparameters through a series of preliminary experiments. The hyperparameters used during the training process are detailed in Table 3. It is worth noting that all the hyperparameters remain fixed during the training process.

5.2 Evaluation indicators

To quantitatively evaluate the quality of task completion and compare the effectiveness of different algorithms, we have

Table 3 Training parameters

Hyper parameters	Values
Training episode	2000
Discount factor γ	0.99
Soft update factor τ	0.005
Critic learning rate	0.0003
Actor learning rate	0.0003
Reply buffer size	1×10^6
Batch size	256
Reward function coefficient η	-0.2
Reward function coefficient β	0.1
Reward function coefficient ψ	-0.2
Reward function coefficient σ	-3

Table 4 Quantitative evaluation indicator (assuming that the number of test rounds is N)

Evaluation indicators	Descriptions
Success rate(SR)(%)	The probability of UAV from origin to target without collision
Collision rate(CR)(%)	The probability of collision between UAV and obstacle
Lost rate(LR)(%)	The probability that UAV crosses the boundary or does not reach the target before the end of the episode
Time cost(TC)	The total time step of the UAV in successful episodes
Average reward(AR)	The average reward for UAV throughout the test

designed a series of quantitative evaluation metrics with a focus on safety, efficiency, and robustness. These metrics are detailed in Table 4.

5.3 Baselines

In order to validate the effectiveness of our proposed algorithm, it is necessary to conduct reasonable comparative experiments. Hence, we choose some proper state-of-the-art Actor-Critic algorithm as baselines.

(1)TD3 [39]. An improved version of the DDPG, which uses double networks with delayed updates and policy smoothing of target networks, resulting in more stable and efficient performance on continuous control tasks.

(2)PPO [40]. A popular policy optimization method that improves training stability and policy performance by defining a trust region to limit the size of updates.

(3)SAC [33]. An entropy-maximizing deep reinforcement learning algorithm that simultaneously optimizes for expected return and policy entropy, leading to more robust policies.

(4)GRU-SAC. Employing GRU to capture and process dynamic feature information, improving performance on tasks with temporal dependencies.

(5)MC-SAC(SAC with multi-channel). Enhancing SAC by using a three-channel fusion mechanism [28] to improve feature extraction capabilities, resulting in better performance in complex environments.

(6)TS-SAC(SAC with two stream network structure). Incorporating a two-stream network structure [29] into SAC to better capture and process multi-type input data features, thereby enhancing policy performance.

Firstly, we can verify that SAC demonstrates a superior exploration capability in complex environments among original AC-based DRL algorithms. Secondly, we incorporate the multi-channel fusion mechanism and the two-stream network structure into SAC and use them as baselines. Additionally, GRU-SAC is used as an ablation model. Thus, we can verify the effectiveness and robustness of our proposed algorithm from multiple perspectives. It is worth noting that, during algorithm training, aside from necessary adjustments, the overall network architecture, network parameter scales, and

hyperparameters of the baselines are kept consistent with those of our proposed algorithm.

5.4 Training and results

In the training phase, we establish an obstacle environment with the parameters $OBS_NUM=20$, $OBS_V=0$, $OBS_R=30$, $OBS_L=50$. This corresponds to a simulation environment where 20 circular or square static obstacles (either with a radius of 30 or a side length of 50) are randomly generated. Additionally, the starting point and target are also randomly generated within the corresponding areas. For each episode, the task is deemed complete if the UAV reaches the target in due time. However, the task fails if the UAV collides with an obstacle or if the time step exceeds the maximum limit. To smoothen the reward convergence curve for a more intuitive analysis of the training progress, we take the rewards from the preceding 100 episodes every ten episodes. The formula for the average reward (AR) is:

$$(Average\ Reward)_i = \begin{cases} \frac{\sum_{j=1}^i Reward_j}{i}, & 0 < i < 100 \\ \frac{\sum_{j=i-99}^i Reward_j}{100}, & i \geq 100 \end{cases}, i = 1, \dots, \frac{Training\ episode}{10}. \quad (15)$$

The convergence curves of the average reward for each algorithm, after 2000 training rounds, are depicted in Fig. 8.

At the beginning of training, the UAV has not accumulated enough experience in the experience replay buffer and can only explore the environment by randomly sampling the action space, so the average reward is quite low. After accumulating sufficient experience, the UAV begins to update the neural network through experience replay. All algorithms start to yield higher average rewards at about 100-episode mark except TD3 and PPO.

Specifically, with regard to the original algorithms, both TD3 and PPO exhibit inadequate exploration capabilities in complex environments. This insufficiency hinders the accumulation of sufficient effective experiences during the initial stages of training, resulting in significantly poorer performance compared to SAC, and even leading to model non-convergence. While original SAC can achieve relatively high average rewards in the training phase, its training pro-

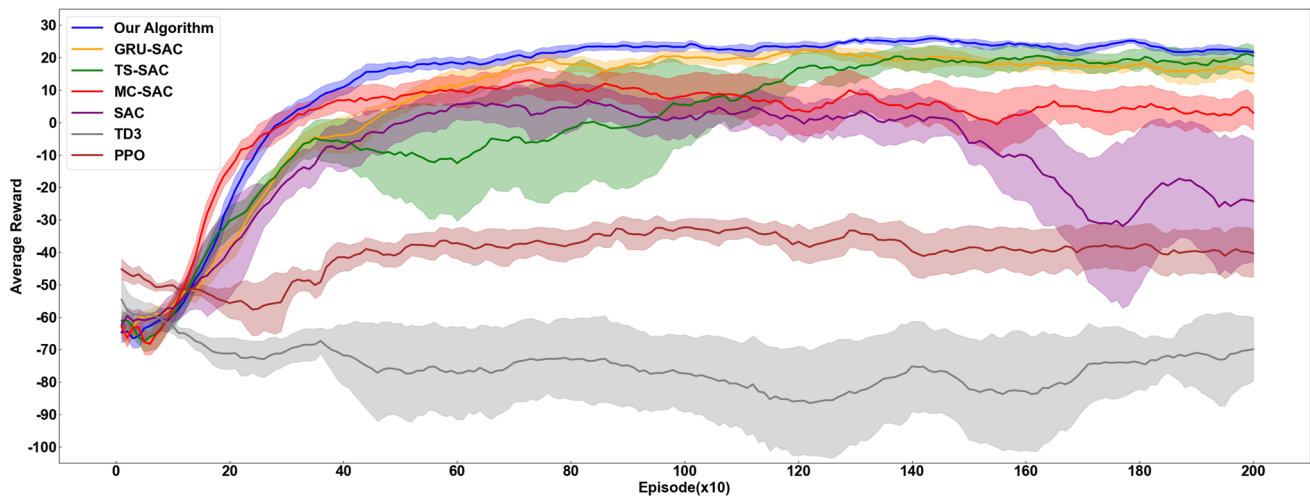


Fig. 8 Convergence curves of average reward obtained from the simulation environment (Each algorithm is randomly trained for 10 times. The solid line represents the mean value, while the shaded area indicates the half standard deviation above and below this mean value)

cess remains unstable and exhibits significant fluctuations during the later stages of training. Building on this foundation, MC-SAC effectively enhances the average reward and stability of training through multi-dimensional feature extraction of the current state, while also improving the convergence speed. Due to the incorporation of historical state information, both GRU-SAC and TS-SAC achieved higher average rewards compared to SAC and MC-SAC, with the average rewards leveling off in the later stages. However, TS-SAC, relying on the introduction of handcrafted temporal features, increases the scale of the state space, resulting in significant fluctuations in the early stages of training and slower convergence. Compared to all the baselines, our algorithm converges at a faster pace and yields a higher maximum average reward. Furthermore, from the shaded area depicting the standard deviation across 10 random trainings, it's evident that the average reward of our algorithm fluctuates less noticeably amid random training.

5.5 Environment generalization test

After training 2000 episodes in the environment with the same environment parameters setting, we engineer three different environment changes to test and evaluate the generalization capabilities of each algorithm. This test focuses on alterations to the number, size, and speed of the obstacles. To ensure a comprehensive comparison of generalization in complex environments, we set the number of test episodes N to 2000. In the test phase, the algorithm model does not consider any stochastic strategy when predicting actions, but selects the action with the highest probability in the action sampling to ensure the consistency of the action and obtain the maximum expected reward as much as possible. Since

TD3 and PPO hardly converge during training, we exclude them from the testing process.

5.5.1 Environment I: More obstacles

In this section, our aim is to test the generalization of different models to varying numbers of obstacles. Accordingly, we fix the environment parameters at $OBS_V=0$, $OBS_R=30$, $OBS_L=50$, while adjusting OBS_NUM to 22, 24, 26, 28, and 30. This approach allows us to evaluate the adaptability of models trained by each algorithm to environments densely populated by obstacles. After 2000 episodes of testing, the test results of the baselines and our algorithm are shown in Fig. 9.

Figure 9 demonstrates that our algorithm consistently outperforms the others, achieving the highest success rate and average reward across all obstacle number tests. As the OBS_NUM value increases and the environment becomes denser, the success rates of all the algorithms show a downward trend. SAC's success rates are the lowest in every test and also have the largest drop (18.55%). MC-SAC is the second lowest success rate in all tests and drops to 75.50% when $OBS_NUM=30$. On the other hand, Our Algorithm, GRU-SAC, and TS-SAC, which take historical environment information into consideration, are all superior to MC-SAC and SAC in terms of success rate. Moreover, these three exhibit higher collision rates, but they have lower lost rates and lower time costs. This pattern suggests that in more complex and dense environments, the decision-making of MC-SAC and SAC, which relies solely on current moment information, tends to be more conservative. By contrast, our algorithm, GRU-SAC and TS-SAC, which take into account temporal motion information, opt for bolder actions to navigate challenging environments. Building upon this, our algorithm, by efficiently capturing valuable historical state

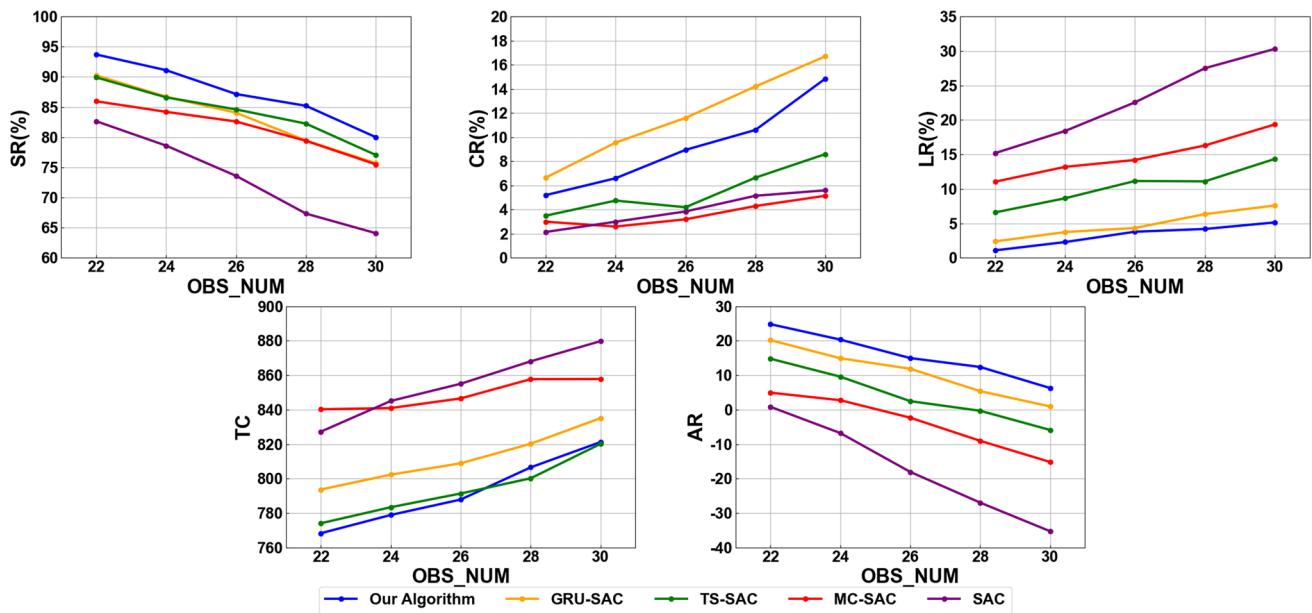


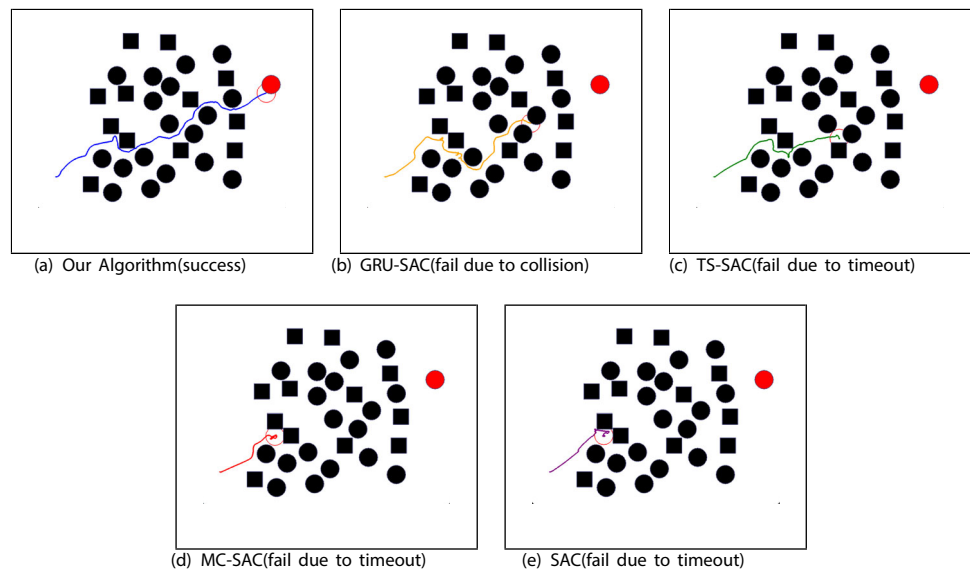
Fig. 9 Changing curves of various evaluation indicators under different OBS_NUM . The horizontal coordinates denote the number of obstacles, and the vertical coordinates denote the evaluation indicators

information while staying attentive to the current state, is able to make more advisable decisions than its ablated version, GRU-SAC, which exhibits an overly aggressive behavior pattern, and TS-SAC, which only introduces information from the previous time step. Therefore, it delivers a higher success rate and average reward than GRU-SAC and TS-SAC.

Figure 10 shows the typical task executions of various algorithms in the same dense environment of this section, where OBS_NUM equals 30. In Fig. 10(a), the UAV employing our algorithm successfully bypasses the concave-like obstacle area at the beginning of the mission and then navigates through the narrow slit, taking the short-

est possible path and successfully reaching the target. This behavior demonstrates that our algorithm can extract valuable information from the historical environment, effectively overcoming the target obstacle dilemma. Moreover, it continues to prioritize real-time information, maintaining a safe distance from obstacles to ensure smooth navigation and successful task completion. On the other hand, as Fig. 10(b) reveals, the UAV utilizing GRU-SAC also tries to cross the narrow slit upon meeting dense obstacles. However, it fails to opportunely adjust its speed and has a tortuous path due to a lack of sensitivity to the current environment information, which ultimately results in task failure due to collision. As

Fig. 10 Typical cases of UAV using various algorithms to perform the same task when $OBS_NUM=30$ (Trajectories are depicted by lines of different colors). Compared to other algorithms, our algorithm navigates dense obstacle clusters more easily



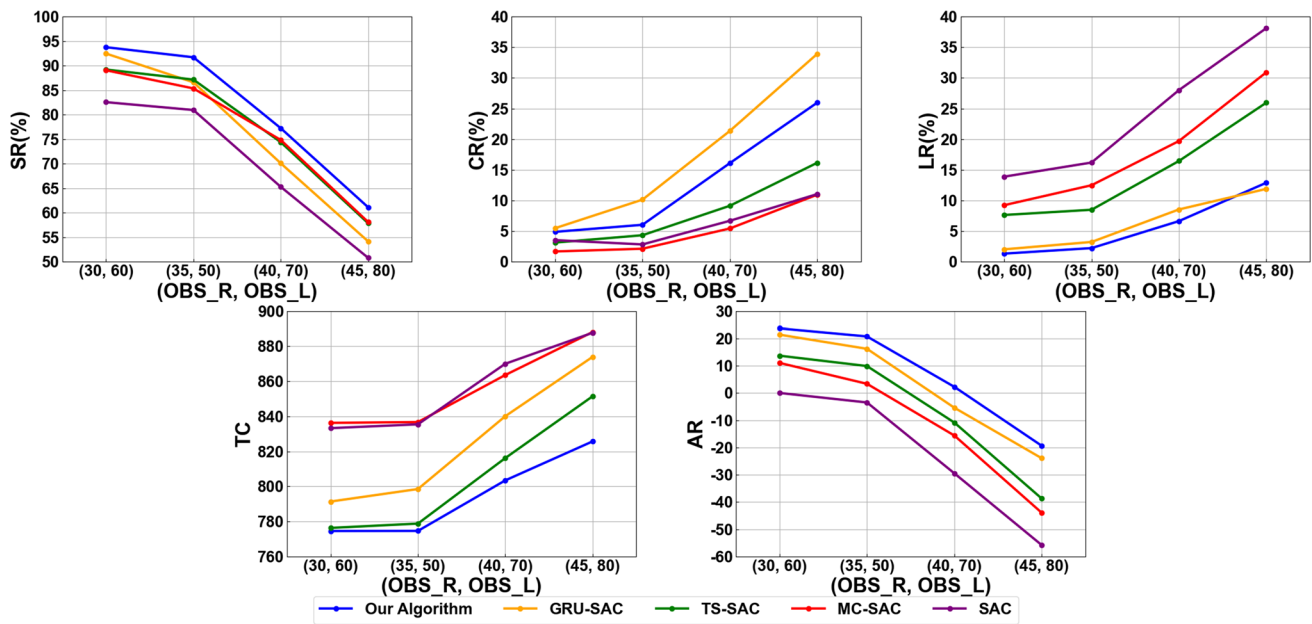


Fig. 11 Changing curves of various evaluation indicators under different (OBS_R, OBS_L) . The horizontal coordinates denote the size of obstacles, and the vertical coordinates denote the evaluation indicators

for the UAV employing TS-SAC as depicted in Fig. 10(c), consideration of the previous state helps it escape a concave-like obstacle area in the early stage of the mission, but then it still gets stuck between obstacles, which causes the mission to time out and fail. This shows that in a dense obstacle environment, simply introducing handcrafted information from the previous time step is not enough to enable the UAV to better traverse obstacles and reach the destination. Lastly, Fig. 10(d) and (e) demonstrate the strategies employed by UAVs utilizing MC-SAC and SAC, which only factor in current-moment environment information. These two decision-making strategies both have excessively conservative behavior patterns. More specifically, not only does the UAV rigidly maintain its distance from the obstacles, leading to a detour, but it also tends to hover between obstacles for a long time. This inefficient flight pattern will cause an increase in task execution time and the freezing robot problem, leading to task failure due to timeout.

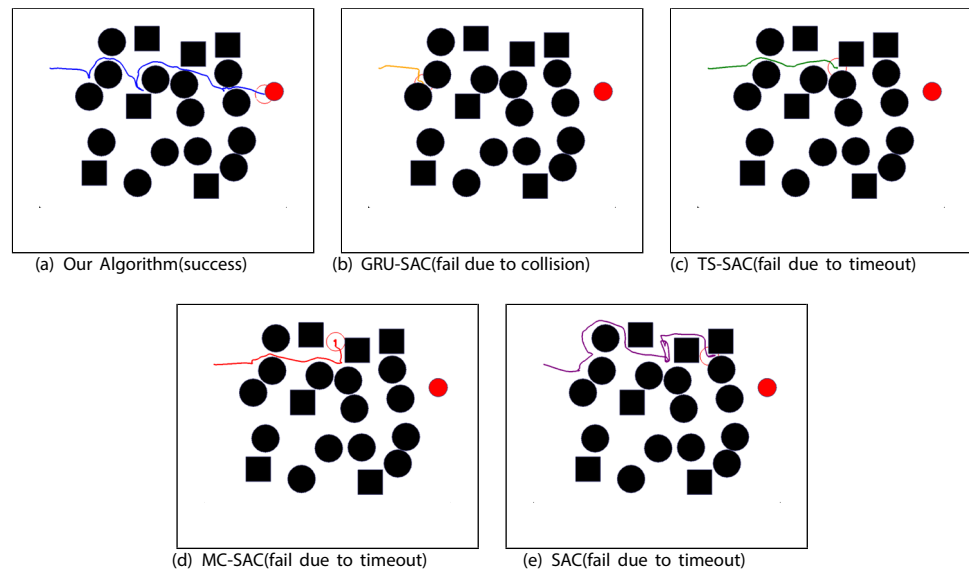
5.5.2 Environment II: Larger obstacles

In this section, we maintain the environment parameters $OBS_V=0$ and $OBS_NUM=20$ unchanged, and set up four groups of larger and different OBS_R and OBS_L , namely $(OBS_R, OBS_L)=(30,60)$, $(35,50)$, $(40,70)$, and $(45,80)$, to test the generalization of different models for larger obstacles. At the same time, larger obstacles are more likely to form concave-like obstacle areas, so this test can also evaluate the ability of each model to break away from the target obstacle dilemma. After 2000 episodes of testing, the results are shown in Fig. 11.

As we can see in Fig. 11, with the size of the obstacle gradually becoming larger, the navigational space for the UAV is squeezed, and the feasible path becomes narrower. Moreover, larger adjacent obstacles contribute to the formation of concave-like environments, which in turn reduces the success rate of autonomous navigation tasks performed by UAVs across all the algorithms. In terms of success rate, original SAC still has the worst performance in this test environment, while its time cost is also the highest. As an ablated version of our algorithm, the shortcomings of GRU-SAC's overly aggressive behavior pattern become apparent as the obstacle size increases. When $(OBS_R, OBS_L)=(30,60)$, its success rate is 92.45%, second only to our algorithm, but as the size of obstacles increases, its collision rate rises sharply and its success rate drops sharply. When $(OBS_R, OBS_L)=(45,80)$, its success rate is only 54.20%, only better than SAC. TS-SAC and MC-SAC are better than GRU-SAC in terms of success rate, but MC-SAC, which only makes better use of the current state information, cannot overcome the high time cost, and the behavior pattern of TS-SAC, which only considers the information of the previous time step, is still relatively conservative compared with our algorithm. In this context, our algorithm consistently performs better than the others, achieving the highest success rate and average reward in all tests, while maintaining the shortest time cost. This suggests that our algorithm is better able to achieve a balance between aggressive and conservative behavior patterns because it can adaptively pay attention to key information in both historical and current states.

Figure 12 illustrates the typical task executions of various algorithms in the same environment when $(OBS_R,$

Fig. 12 Typical cases of UAV using various algorithms to perform the same task when $(OBS_R, OBS_L)=(45, 80)$ (Trajectories are depicted by lines of different colors). Compared to other algorithms, our algorithm navigates through large obstacles or escapes the target obstacle dilemma more easily



$OBS_L=(45,80)$. In Fig. 12(a), the UAV employing our algorithm hovers and explores the surrounding environment for a while upon encountering a concave-like obstacle area caused by the clustering of large obstacles, and then successfully escapes from the area. Moreover, in the later stage of the task, it not only opts to navigate through the narrow slit, but also executes the process with remarkable stability, ultimately achieving successful task completion. Conversely, in Fig. 12(b), the UAV utilizing GRU-SAC also attempts to traverse the slit when encountering the concave-like area but collides with obstacles during the process, resulting in task failure. Notably, we can observe that the UAV tries to pass through the slit as soon as it senses it, with little regard for the risk. This can be attributed to GRU-SAC's inability to maintain sensitivity to crucial current information through historical context, leading to a delay in obstacle avoidance and eventual collision. In Fig. 12(c), the UAV using TS-SAC can maneuver between the large obstacles with noticeable slits. Nevertheless, when faced with these sizable obstacles clustering tightly to form narrow slits, the UAV can only hovers between obstacles, falling into the target obstacle dilemma and freezing robot problem, thus stuck between the obstacles until the task fails due to timeout. In Fig. 12(d), the path of the UAV employing MC-SAC, which performs multi-channel feature extraction on the current state, is basically the same as that of TS-SAC. However, its behavior pattern is more conservative. As we can see, the UAV chooses to bypass the same narrow slit as TS-SAC meets, but it still ends up freezing and timing out. As for the SAC-guided UAV path, we can find that it is very tortuous and conservative. The UAV as depicted in Fig. 12(e) immediately chooses to take a detour once it senses a relatively narrow space, and it is very easy to get stuck in the target obstacle dilemma and the

freezing robot problem. As expected, it eventually fails due to timeout when it gets stuck in a concave-like obstacle area.

5.5.3 Environment III : Dynamic obstacles

Finally, in order to evaluate the perception and dynamic avoidance capabilities of different models in response to the relative motion trends of obstacles, we maintain the obstacle environment parameters as $OBS_NUM=20$, $OBS_R=30$, $OBS_L=50$, and vary OBS_V from 1 to 5, representing the initial speed of all obstacles with the ability to freely collide and a collision coefficient of 1. It should be noted that OBS_V only determines the magnitude of the initial velocity of the obstacle, while its direction is randomly generated when the environment is initialized, which fully guarantees the randomness of the dynamic environment. Moreover, in order to ensure that obstacles always remain within the task space and to avoid interfering with the generation process of the starting point and target in the early stages of the task, we specify that when obstacles reach the boundaries of the UAV's starting and target areas, a fully elastic collision with a coefficient of 1 also occurs. After 2000 episodes of testing, the results of each algorithm are presented in Fig. 13.

Figure 13 illustrates that with the obstacle speed increasing, the task environment becomes more complex, and the task success rates of all the algorithms decrease to varying degrees. For slow obstacle speeds ($OBS_V=1$ or $OBS_V=2$), the decline is subtle because the slight obstacle movement allows the UAV ample time to maneuver, it is not an urgent need to judge whether the obstacle is approaching or moving away from the UAV. However, as the obstacle speed continues to increase, the environment becomes

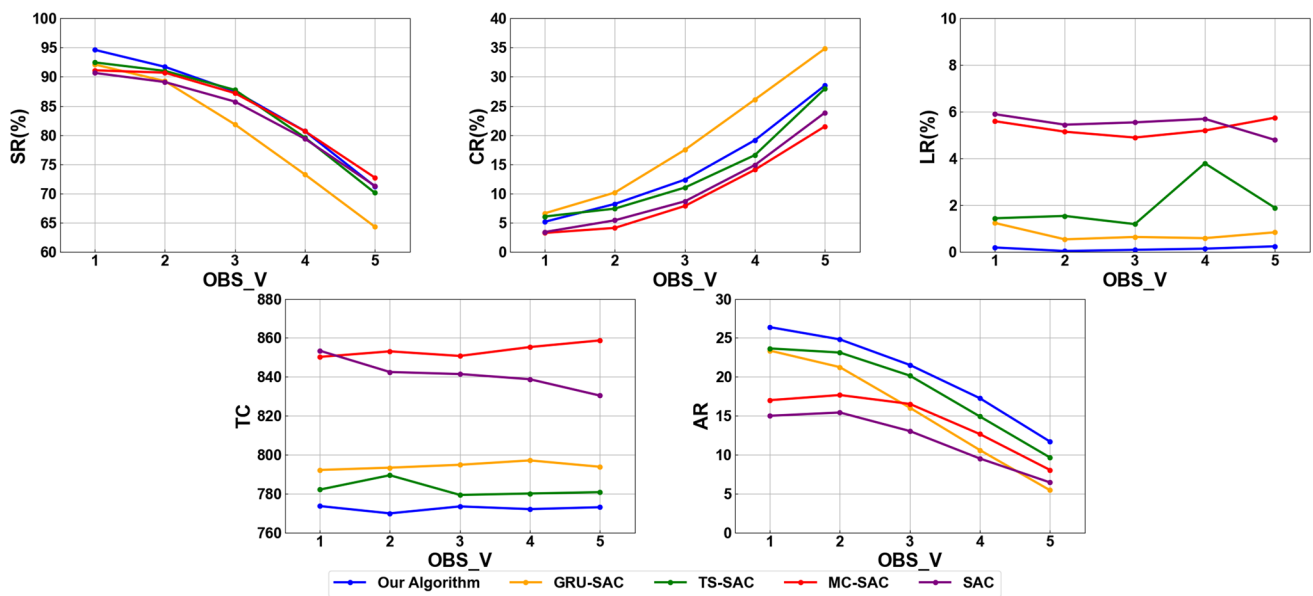


Fig. 13 Changing curves of various evaluation indicators under different OBS_V . The horizontal coordinates denote the magnitude of the initial velocity of obstacles of obstacles, and the vertical coordinates denote the evaluation indicators

very dynamic and complex, in which case the performance of all the algorithms has decreased significantly. Among them, GRU-SAC has the lowest success rate of 64.35% at $OBS_V=5$, which is markedly lower than the other four algorithms, and the collision rate is also significantly higher than the other four. This highlights that while GRU-SAC leverages historical information and relative obstacle motion trends to some extent, it tends to overlook current moment details, leading to a radical behavior pattern that delays obstacle avoidance, especially evident in high-speed obstacle environment. TS-SAC performs better than GRU-SAC in dynamic environments because it only pays attention to the current state and the state of the previous time step. However, it cannot account for more distant historical motion information while being sensitive to the current state, so its overall performance across various indicators is inferior to our algorithm. It is worth noting that while MC-SAC and SAC have the worst performance in low-speed obstacle environments, as obstacle speed rises, their success rates aligns closely with the other algorithms and even slightly outperforming our algorithm at $OBS_V=5$ (At this time, the success rates of MC-SAC, SAC, and our algorithm are 72.75%, 71.35%, and 70.15%, respectively). This is because the action selection of SAC at the current moment is too conservative. This behavior pattern is easy to make the UAV fall into the target obstacle dilemma and freezing robot problem in a denser and larger obstacle environment, but in an environment where the obstacle speed is very fast, it avoids the occurrence of collision to a certain extent. On the other hand, the rapid movement of obstacles also makes it difficult for the task environment to form fixed concave-like obstacle areas, which also further

improves the success rate. However, despite matching our algorithm in success rate in high dynamic obstacle environments, MC-SAC's and SAC's task completion efficiency still lags significantly behind. When $OBS_V=5$, the time cost of MC-SAC and SAC to successfully complete the task is 85.58 and 57.28 time steps longer than that of our algorithm, which further shows that our algorithm can pay attention to the current situation of the UAV while capturing the key information in the historical state, so that the UAV can not only escape away from the complex obstacle environment, but also complete the task smoothly, safely and efficiently.

6 Conclusions

In this paper, we propose a novel deep reinforcement learning approach based on temporal attention to facilitate real-time navigation and obstacle avoidance for UAVs in dense, dynamic, and complex environments. Our method, building upon the SAC algorithm, integrates temporal changes in the state space using a temporal attention mechanism to encode historical information of the environment. This enables the UAV to adaptively extract key historical information while remaining sensitivity to real-time environment information. In order to verify the effectiveness of our algorithm, we establish a visualized two-dimensional continuous simulation platform and a set of reward functions suitable for guiding the UAV to complete the navigation task. Then, we set the original algorithms PPO, TD3, SAC, the ablated version of our algorithm GRU-SAC, and TS-SAC, MC-SAC which are the combination of state feature extraction meth-

ods proposed by other researchers and SAC, as baselines and train them alongside the proposed algorithm respectively. The results show that our algorithm exhibits more stable convergence and higher average reward during the training phase. Additionally, we design different simulation tests to evaluate and compare the performance and environment generalization of each algorithm. Through testing in different simulation environments, we find UAVs employing SAC demonstrate a highly conservative task execution approach. MC-SAC, which uses a multi-channel mechanism to optimize the way current state information is processed, slightly improves this shortcoming, but its overall performance is still unsatisfactory and prone to target obstacle dilemmas and freezing robot problems because it does not consider temporal information. As an ablated version of our algorithm, GRU-SAC takes into account historical information from the previous period of time to alleviate the target obstacle dilemma, but it exhibits an overly aggressive behavior pattern due to insufficient attention to the current state, which makes it very easy to fail the task due to collisions in complex environments. In comparison, TS-SAC, which only introduces the state information of the previous time step, can achieve a balance between radicalism and conservatism in most conditions and achieve a higher success rate. Compared to all the baselines, our algorithm achieves the highest success rates, average rewards, and the minimal time cost in almost all the test environments. This indicates that our algorithm can not only capture crucial information from historical states, but also be attuned to the UAV's current situation, enabling the UAV to avoid obstacles and reach the target smoothly, safely, and efficiently.

Although this paper has verified the effectiveness and robustness of the proposed algorithm in detail through extensive comparative experiments. However, these are all methods of autonomous navigation that react to raw data based on infrared detectors, focusing mainly on local and immediate navigation decisions, and unable to optimize paths over a larger range. This natural defect limits the model's long-term or even global memory capabilities. At the same time, although the infrared detector can provide distance information of obstacles, its perception ability is limited and it cannot provide rich environmental details and contextual information. This limitation may limit the performance of UAVs in real environments.

In future work, we will consider solving more complex and realistic UAV autonomous navigation tasks. Firstly, we will build a 3D simulation environment with higher accuracy to narrow the slit between the algorithm training environment and the real world. Secondly, richer and more realistic sensor information, such as images obtained by monocular cameras will be considered. We will further improve the UAV's perception of the environment through multimodal data fusion, and try to integrate it with technologies such as map building

to improve the UAV's ability to remember the global environment. Finally, we plan to add a more physically accurate aerodynamic model to the environment, such as considering the effect of complex turbulence on the flight performance of UAVs.

Appendix A. Notation table

Notation	
A	Action space
a_t	The action taken by the agent at time t
abs_x	The x-coordinate of the UAV with the lower left corner of the environment as the origin
abs_y	The y-coordinate of the UAV with the lower left corner of the environment as the origin
$ActorLoss$	Loss function of the Actor
$CriticLoss$	Loss function of the Critic
d_{min}	The minimum distance between the UAV and the obstacle in the whole navigation task
$detected_distance$	The length of the ray when the infrared detector detects an obstacle
F	The magnitude of the thrust applied to the UAV
J	Objective function of the Actor
L	Objective function of the Critic
o_j	The length of the j th infrared ray
$obsInfo$	The information about obstacles measured in by the infrared detector
P	State transition probability function
p_t	The UAV's position in the environment at time t
Q	Action-value function
$Q(s_t, a_t)$	The action-value that an agent receives after making action a_t in state s_t
R	Reward function
r_t	The reward that an agent receives at time t
$r_{distance}$	Distance reward
$r_{shaping}$	Shaping reward
$r_{avoidance}$	Avoidance penalty
$r_{boundary}$	Boundary penalty
ref_x	The x-coordinate of the UAV with the target as the origin
ref_y	The y-coordinate of the UAV with the target as the origin
S	State space
s_t	The state of the agent at time t
$selfInfo$	The information about the UAV measured by GPS and gyroscope
vel_x	The speed of the UAV in the x-axis direction
vel_y	The speed of the UAV in the y-axis direction
$y(r_t, s_{t+1})$	The TD target of the experience (s_t, a_t, r_t, s_{t+1})
α	Temperature coefficient
β	The coefficient of shaping reward
γ	Discount factor
η	The coefficient of distance reward
θ	The direction of the thrust applied to the UAV
π	Strategy of the agent
σ	The coefficient of avoidance penalty
ψ	The coefficient of avoidance penalty

Supplementary Information The online version contains supplementary material available at <https://doi.org/10.1007/s10489-024-06036-2>.

Author Contributions Shuyuan Liu contributed to the conception of this research and performed the experiment; Xinghua Chang and Laiping Zhang contributed significantly to analysis and manuscript preparation; Shufan Zou, Huayong Liu and Xiaogang Deng contributed to perform the analysis with constructive discussions. All authors have read and agreed to the published version of the manuscript.

Funding This research is funded by the National Key Project of China (Grant No. GJXM92579) and Sichuan science and Technology Program (Grant No. 2023YFG0329).

Data availability and access All data, models, or code generated or used during the study are available from the corresponding author by reasonable request.

Declarations

Ethical and informed consent for data used There is no ethical conflict and no need to informed consent for data.

Competing interests There are no competing interests to declare.

References

- Xu L, Wang T, Cai W, Sun C (2023) Uav target following in complex occluded environments with adaptive multi-modal fusion. *Appl Intell* 53(13):16998–17014
- Rezwani S, Choi W (2022) Artificial intelligence approaches for uav navigation: recent advances and future challenges. *IEEE Access* 10:26320–26339
- Yanmaz E (2023) Joint or decoupled optimization: multi-uav path planning for search and rescue. *Ad Hoc Netw* 138:103018
- Castellano G, Castiello C, Mencar C, Vessio G (2020) Crowd detection in aerial images using spatial graphs and fully-convolutional neural networks. *IEEE Access* 8:64534–64544
- Li W, Li S, Shi H, Yan W, Zhou Y (2024) Uav-enabled fair offloading for mec networks: a drl approach based on actor-critic parallel architecture. *Appl Intell* 54(4):3529–3546
- Saccani D, Cecchin L, Fagiano L (2022) Multitrajectory model predictive control for safe uav navigation in an unknown environment. *IEEE Trans Control Syst Technol* 31(5):1982–1997
- Sajid M, Mittal H, Pare S, Prasad M (2022) Routing and scheduling optimization for uav assisted delivery system: a hybrid approach. *Appl Soft Comput* 126:109225
- Guo T, Jiang N, Li B, Zhu X, Wang Y, Du W (2021) Uav navigation in high dynamic environments: a deep reinforcement learning approach. *Chin J Aeronaut* 34(2):479–489
- AlMahamid F, Grolinger K (2022) Autonomous unmanned aerial vehicle navigation using reinforcement learning: a systematic review. *Eng Appl Artif Intell* 115:105321
- Zhu K, Zhang T (2021) Deep reinforcement learning based mobile robot navigation: a review. *Tsinghua Sci Technol* 26(5):674–691
- He Z, Liu C, Chu X, Negenborn RR, Wu Q (2022) Dynamic anti-collision a-star algorithm for multi-ship encounter situations. *Appl Ocean Res* 118:102995
- Huang T, Fan K, Sun W (2024) Density gradient-rrt: an improved rapidly exploring random tree algorithm for uav path planning. *Expert Syst Appl* 252:124121
- Pehlivanoglu YV, Pehlivanoglu P (2021) An enhanced genetic algorithm for path planning of autonomous uav in target coverage problems. *Appl Soft Comput* 112:107796
- Fan J, Chen X, Wang Y, Chen X (2022) Uav trajectory planning in cluttered environments based on pf-rrt* algorithm with goal-biased strategy. *Eng Appl Artif Intell* 114:105182
- Phung MD, Ha QP (2021) Safety-enhanced uav path planning with spherical vector-based particle swarm optimization. *Appl Soft Comput* 107:107376
- He W, Qi X, Liu L (2021) A novel hybrid particle swarm optimization for multi-uav cooperate path planning. *Appl Intell* 51(10):7350–7364
- Yang H, Xu X, Hong J (2022) Automatic parking path planning of tracked vehicle based on improved a* and dwa algorithms. *IEEE Trans Transp Electr* 9(1):283–292
- Lee S, Lee H (2022) Trajectory generation of a quadrotor transporting a bulky payload in the cluttered environments. *IEEE Access* 10:31586–31594
- Pan Z, Zhang C, Xia Y, Xiong H, Shao X (2021) An improved artificial potential field method for path planning and formation control of the multi-uav systems. *IEEE Trans Circuits Syst II Express Briefs* 69(3):1129–1133
- Wang H-n, Liu N, Zhang Y-y, Feng D-w, Huang F, Li D-s, Zhang Y-m (2020) Deep reinforcement learning: a survey. *Front Inf Technol Electron Eng* 21(12):1726–1744
- Ladosz P, Weng L, Kim M, Oh H (2022) Exploration in deep reinforcement learning: a survey. *Inf Fusion* 85:1–22
- Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, Graves A, Riedmiller M, Fidjeland AK, Ostrovski G et al (2015) Human-level control through deep reinforcement learning. *nature* 518(7540):529–533
- Wang X, Wang S, Liang X, Zhao D, Huang J, Xu X, Dai B, Miao Q (2022) Deep reinforcement learning: a survey. *IEEE Trans Neural Netw Learn Syst* 35(4):5064–5078
- Huang H, Yang Y, Wang H, Ding Z, Sari H, Adachi F (2019) Deep reinforcement learning for uav navigation through massive mimo technique. *IEEE Trans Veh Technol* 69(1):1117–1121
- Moon J, Papaioannou S, Laoudias C, Kolios P, Kim S (2021) Deep reinforcement learning multi-uav trajectory control for target tracking. *IEEE Internet Things J* 8(20):15441–15455
- Wang C, Wang J, Wang J, Zhang X (2020) Deep-reinforcement-learning-based autonomous uav navigation with sparse rewards. *IEEE Internet Things J* 7(7):6180–6190
- Zhang L, Peng J, Yi W, Lin H, Lei L, Song X (2023) A state-decomposition ddpg algorithm for uav autonomous navigation in 3d complex environments. *IEEE Internet Things J*
- Li B, Huang Z, Chen TW, Dai T, Zang Y, Xie W, Tian B, Cai K (2022) Msn: mapless short-range navigation based on time critical deep reinforcement learning. *IEEE Trans Intell Transp Syst* 24(8):8628–8637
- Zhang S, Li Y, Dong Q (2022) Autonomous navigation of uav in multi-obstacle environments based on a deep reinforcement learning approach. *Appl Soft Comput* 115:108194
- Xue Y, Chen W (2022) A uav navigation approach based on deep reinforcement learning in large cluttered 3d environments. *IEEE Trans Veh Technol* 72(3):3001–3014
- Xue Y, Chen W (2023) Combining motion planner and deep reinforcement learning for uav navigation in unknown environment. *IEEE Robot Autom Lett* 9(1):635–642
- Botvinick M, Wang JX, Dabney W, Miller KJ, Kurth-Nelson Z (2020) Deep reinforcement learning and its neuroscientific implications. *Neuron* 107(4):603–616
- Haarnoja T, Zhou A, Abbeel P, Levine S (2018) Soft actor-critic: off-policy maximum entropy deep reinforcement learning with a

- stochastic actor. In: International conference on machine learning. PMLR, pp 1861–1870
34. Sutton RS, McAllester D, Singh S, Mansour Y (1999) Policy gradient methods for reinforcement learning with function approximation. *Adv Neural Inf Process Syst* 12
 35. Van Hasselt H, Guez A, Silver D (2016) Deep reinforcement learning with double q-learning. In: *Proceedings of the AAAI conference on artificial intelligence*, vol 30
 36. Fan J, Wang Z, Xie Y, Yang Z (2020) A theoretical analysis of deep q-learning. In: *Learning for dynamics and control*. PMLR, pp 486–489
 37. Fu C, Xu X, Zhang Y, Lyu Y, Xia Y, Zhou Z, Wu W (2022) Memory-enhanced deep reinforcement learning for uav navigation in 3d environment. *Neural Comput Appl* 34(17):14599–14607
 38. Sathiamoorthy AJ, Patel U, Guan T, Manocha D (2020) Frozone: freezing-free, pedestrian-friendly navigation in human crowds. *IEEE Robot Autom Lett* 5(3):4352–4359
 39. Fujimoto S, Hoof H, Meger D (2018) Addressing function approximation error in actor-critic methods. In: *International conference on machine learning*. PMLR, pp 1587–1596
 40. Gu Y, Cheng Y, Chen CP, Wang X (2021) Proximal policy optimization with policy feedback. *IEEE Trans Syst Man Cybern Syst* 52(7):4600–4610

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.